SDP



# Module 2: Agent Plan Mode

⏰ Plan the work before the work begins

*"We gave Copilot the context. Now let's see if it actually understands our codebase — by asking it to plan real work."*

M1 Instructions › **M2 Plan Mode** › M3 Prompts › M4 Skills › M5 MCP › M6 Agents

⏰ : Monday, 11:30 AM

M **Marcus** Sees the next feature request and realizes the foundation is no longer the problem.

> "Wait — it just referenced ARCHITECTURE.md without me asking. When did that start working? Okay... so what happens when I actually plan something?"

**Module 0**

### Feel the mess

The team experienced weak context and inconsistent defaults firsthand.

**Module 1**

### Build the foundation

Architecture docs and instructions created a better baseline for Copilot behavior.

**Module 2**

### Plan the work

Now the question is what to change, in what order, and how to challenge the roadmap before coding.

⚡ **Plan mode turns "Where do I start?" into a reviewable roadmap before code starts landing.**

## The Team, Before Module 2

**WRONG ANSWERS ONLY**

### Jessica
*The Eager Junior*

*"Plan mode just writes the code more slowly so you can check it as it goes. It's like watching it think out loud."*

About to discover the plan is the point.

### David
*The Seasoned Architect*

*"Plan mode outputs a structured diagram of the proposed changes. I've been asking for that for two years."*

Correctly excited. Completely wrong about what it outputs.

### Sarah
*The Skeptical Senior*

*"It's read mode. It reads the code before writing instead of after. Like a junior dev who actually does that."*

The bar she set is doing a lot of work here.

### Elena
*The Quality Champion*

*"Plan mode means it won't write any code until you explicitly say go. It just waits. That's the feature."*

Wrong about the mechanism. Right about wanting control.

### Marcus
*The DevOps Developer*

*"Plan mode is when it shows you the diff before applying it. I thought I was already using that."*

Mixing up three different features with confidence.

### Rafael
*The Product Visionary*

*"Plan mode auto-generates the ticket description. Marcus told me. That's what we're doing today, right?"*

Marcus did not tell him that.

🎯 What You'll Build

## From feature request to step-by-step execution roadmap.

One feature, three exercises, one calmer path to implementation

### 2.1–2.2 — Research-backed plans

`/plan mode`

Two first-pass roadmaps with likely files, dependencies, and open questions surfaced before coding.

### 2.3 — Cleaner execution

`Agent mode handoff`

Hand the validated roadmap to Agent mode — implementation with less guesswork and fewer conflicts.

💡 A good plan changes how you start — you know which files to touch, in what order, before the first change lands.

## What Plan Mode Actually Does

### Researches read-only first

Reads and maps before touching code — never modifies your files.

- Reads the codebase, docs, and existing patterns
- Maps dependencies and likely file touch points
- Surfaces assumptions and open questions

### Produces a reviewable draft

A structured plan you can read, edit, and approve before any code is written.

- Summary of the approach and scope
- Ordered implementation steps
- Dependencies and unresolved questions

### Builds on Module 1

Your context files become the planning foundation, not just completion hints.

- Uses your architecture context automatically
- Follows your documented conventions
- Understands your show's domain from the universe file

### Still needs human judgment

Reduces discovery work — doesn't replace architectural thinking or team decisions.

- Edge cases and failure modes
- Architecture tradeoffs and tech debt
- Testing completeness and rollout decisions

**Plan mode improves the first draft. It does not replace review.**

## 🔗 The Plan-First Workflow

### 🔍 Research

Ask plan mode to map the work before touching code.

### 👀 Review

Read the proposed steps like a design review, not a final answer.

### 🔄 Refine

Add missing requirements, edge cases, tests, and constraints.

### ✅ Execute

Implement from a clearer roadmap with less thrash.

**Move your thinking earlier — where it changes the outcome instead of the cleanup.**

> *"I've been asking engineering to 'move the thinking earlier' for ten years. Turns out there's a button."*
>
> — Rafael

## ❌ Before vs ✨ After

### Manual analysis eats time

Developers spend 30+ minutes tracing files, imports, and dependencies before making the first change.

### Missing dependencies surface late

Architecture gaps often show up during implementation or PR review instead of during planning.

### Tests and edge cases get bolted on

The happy path gets planned first, while quality and failure modes show up as cleanup work later.

### Research starts faster

A complex feature can move from manual exploration toward a draft plan in minutes instead of half an hour.

### Review happens before implementation

Architectural issues, missing tests, and unclear requirements get challenged while the work is still cheap to change.

### Execution gets calmer

The team codes against a validated roadmap instead of rediscovering the plan in the middle of implementation.

## ✅ What Good Plan Output Looks Like

### A strong draft includes

- A clear task summary
- Ordered implementation steps
- Specific file paths or system areas
- Dependencies and open questions

### A strong review checks

- Error handling and edge cases
- Architectural fit and existing patterns
- Test coverage and affected test files
- Scope assumptions that need clarification

**Good plan mode output is specific enough to challenge and simple enough to refine.**

> *"Specific enough to challenge, simple enough to refine. That's a well-formed specification. I've been asking for those for twenty years."*
>
> — David

## What to Validate in the Workshop

### Planning time

Compare manual feature analysis with plan generation plus review.

### Plan completeness

Check whether file paths, dependencies, and open questions are visible before implementation starts.

### Rework avoided

Look for fewer missed dependencies, earlier test planning, and fewer review surprises.

### Make the improvement observable.

The strongest version of this module shows how planning changes what gets built — and how confidently it gets there.

🔨 Exercises — Plan Mode

## 3 exercises · ~35 minutes · shared leadership

### 2.1 — Plan Lore card Jessica ⭐

Add a Lore card to the homepage and a new Lore page. Build the plan before writing a single line of code.

Practice reviewing & editing the plan before approving it

### 2.2 — Plan add form David ⭐

Produce a second roadmap for the add-episode feature — in the same session as 2.1, without losing context.

One conversation, two independent plans — no rediscovery needed

### 2.3 — Execute + populate Sarah ⭐

Hand the validated plan to Agent mode and let it implement — without re-explaining the codebase from scratch.

The plan is the handoff — execution is just follow-through

*"I kept wanting to just start typing. Making myself write the plan first felt slow. Then I saw how fast the execution went."*

— Sarah

EXERCISE

# 2.1 Plan the Lore Card & Page

Jessica ⭐ | 10 min

The homepage already has cards for Characters, Episodes, Seasons, and Network. Jessica is adding the missing one: a **Lore card** that shows the count of lore facts and navigates to a new Lore page listing them in a card layout. She loves it — visible, interactive, exactly what users notice. **She runs plan mode to map the work before touching code, completely focused on making the display right.**

💭 *"A new page, new route, DB migration — I thought I was just dropping in another card." — Jessica*

## 📋 STEPS

1. Switch to Plan mode — then write down your guess: how many files will this touch?
2. Enter the Lore card prompt and watch plan mode read the codebase (60–120 s)
3. Count files in the plan, compare to your guess, annotate ≥ 1 refinement you'd make
4. Click Open in Editor, then Save As Prompt File (lower-right) — David will spot what's missing in 2.2

## ✅ SUCCESS CRITERIA

- ☐ Plan runs without modifying any file
- ☐ Plan identifies [illegible] DB migration.
- ☐ File-count guess recorded [illegible] plan shows [illegible]
- ☐ [illegible] `.prompt.md`

## Example: The Lore Card Plan Prompt

Use `/plan` at the start of your prompt — this switches to plan mode automatically if you forgot to change the mode selector:

```
/plan Add a Lore card to the FanHub homepage that shows the count of lore facts for the show. Clicking the card navigates to a new /lore page that lists all lore facts in a card layout.
```

**Check the bottom of the plan**

Plan mode often auto-seeds lore data from the universe file you created in M1 — scroll down before moving on

**What to expect**

60–120 seconds of research — without touching any file

**Watch the chat window while it runs.** You'll see Copilot reading `ARCHITECTURE.md`, the existing card components, routing config, and DB schema — each listed as a tool call. It's learning the pattern of the existing cards before proposing how to add a new one.

**Save the plan:** Click `Open in Editor` when the plan finishes. A `Save As Prompt File` button appears — click it to save as a `.prompt.md` file for Exercise 2.2.

## Example: First Plan Output (Lore Card & Page)

```
## Add Lore Card to Homepage + /lore Page

**Scope:** New Lore card on homepage showing lore count; new /lore route listing
entries in card layout

### Step 1: Database

- migrations/add_lore_entries.sql → New `lore_entries` table (id, title, description,
created_at)

### Step 2: API Route

- routes/lore.ts → GET /api/lore-entries (returns all entries + count)

### Step 3: Lore Page Component

- pages/lore.tsx → fetches lore entries, renders each as a card (title + description)

### Step 4: Homepage Card Update

- pages/index.tsx → add Lore card to existing grid, link to /lore, show count from API

### Step 5: Routing

- router/index.ts → register /lore route pointing to lore.tsx
```

Files touched: ~5 Most guessed: 2–3 Gap: this is the lesson

EXERCISE

# 2.2 Plan the Lore Entry Form

David ⭐ | 10 min

Jessica shares her plan output with the group. David reads it line by line — all the way through. The Lore card, the `/lore` page, the migration, the API route. *He notices the plan has no step for adding entries. The table gets created, the page reads from it — but nothing writes to it.* **David opens Jessica's saved plan file in VS Code, then runs a second plan for a Show Lore entry form — VS Code detects the open plan and offers to combine them.**

💭 *"The plan creates the table and reads from it. Nothing writes to it. How do entries ever get in there?" — David*

## 📋 STEPS

1. Open Jessica's plan file in VS Code — having it open is how Copilot detects the existing plan context
2. Run the Show Lore form plan prompt — VS Code will ask to combine or keep plans separate — choose Combine
3. Find the open question the plan surfaces that requires your judgment; write your answer
4. Click Open in Editor, then Save As Prompt File (lower-right) — combined plan ready for execution

## ✅ SUCCESS CRITERIA

- ☐ Jessica's plan file open in VS Code before running the second plan
- ☐ Combined plan references existing table — no duplicate invented
- ☐ POST `/api/lore` endpoint included in combined plan
- ☐ At least one open question surfaced and answered before implement

## Example: The Lore Entry Form Plan Prompt

David enters this with Jessica's plan file open in VS Code:

```
/plan Add a form to the /lore page that lets me add a new lore entry. The form has a
title and description field. On submit, save the new entry and refresh the list so
the new card appears immediately. The Lore count on the homepage card also updates.

Before you create the plan, ask me any clarifying questions — for example: does this
extend the existing lore card plan or become a separate plan? Is the form reachable
from global nav? Any other implementation choices I need to decide before you start.
```

🔍 **Why ask first:** Without this, Copilot picks defaults silently — and you only discover them when the plan is finished. Upfront questions surface decisions like nav placement and whether to extend or replace plan 3 before the plan is written. Once you've answered, run the continuity check: same **table name**, same **route pattern** as `/api/lore-entries`, same **type enum values**.

📋 Example: Second Plan Output (Lore Entry Form)

```
## Add Entry Form to /lore Page

**Scope:** Form on /lore page with title + description fields; on submit, entry saved
and card list refreshes; homepage Lore count updates

### Step 1: API Endpoint

- routes/lore.ts → POST /api/lore-entries (inserts into lore_entries ✓ same table as
Jessica's plan)

### Step 2: Form Component

- pages/lore.tsx → add form above card list; title + description inputs; submit
handler calls POST, then re-fetches list

### Step 3: Homepage Count Sync

- pages/index.tsx → Lore card count re-fetches from GET /api/lore-entries on
navigation back to homepage

### Open Questions (requires your judgment)

- Does the form go above or below the existing lore cards?
- New entry: append to bottom of list, or sort newest-first?
```

Continuity check: references `lore_entries` ✓ Route pattern: matches `/api/lore-entries` from plan 1 ✓

EXERCISE

# 2.3 Execute the Plan and Populate Your Universe

Sarah ⭐ | 15 min

Sarah reviews `{plan}.prompt.md` — two plans merged: Jessica's Lore card and page, David's Show Lore entry form. **She simply runs the prompt file and watches every file change.**

💭 *"After execution, the homepage will have five cards. And the Lore count won't stay at zero." — Sarah*

## STEPS

1. Run /{plan}.prompt.md — execution begins
2. Monitor for deviations — if agent drifts from the plan, flag it and redirect
3. Verify: Show Lore form submits, Show Lore counter increments, Network tab confirms POST to `/api/lore-entries`
4. Add ≥ 3 Show Lore entries from your show — watch the counter climb from 0

## SUCCESS CRITERIA

- [ ] Homepage renders all 5 stat cards
- [ ] Lore form posts correctly and refreshes the Show Lore counter
- [ ] ≥ 3 Show Lore entries added; Show Lore counter reflects correct total
- [ ] Test suite passes; new tests cover Lore endpoints for both Jessica and David

## 📋 Example: Executing the Roadmap

### The execution prompt

*[Switch mode selector to **Agent**]*

**/plan-loreCardAndLorePageForm.prompt**

---

What to watch for:

- Agent creates table named differently → flag it
- Agent adds pagination (not in plan) → decide: accept or revert
- Agent skips a step → point it back to the roadmap

### The payoff — Lore card complete

Homepage before execution:

Characters ✓ · Episodes ✓ · Seasons ✓ · Network ✓
Lore: missing

After execution + adding entries:

Characters ✓ · Episodes ✓ · Seasons ✓ · Network ✓
**Lore: 3 ✓ → /lore page lists them all**

*The plan described exactly what got built — no surprises.*

**The plan → execute arc closes here.** Plan mode made the implementation safe. Your show data makes it yours.

## ✅ Module Checklist

**By the end of Module 2, the team has a planning rhythm.**

Read the plan before touching code. Improve and iterate. Execute with confidence.

### Plan Lore card & page
EXERCISE 2.1

- Research-first before any code
- Guess file count → compare
- Save plan as .prompt.md

### Plan the Lore form
EXERCISE 2.2

- Read the plan — spot the write gap
- Combine with Jessica's plan
- Continuity check: lore_entries ✓

### Execute + populate
EXERCISE 2.3

- Run /{plan}.prompt.md in agent mode
- Verify Lore card, page, and form
- Add ≥ 3 lore entries; count goes live

**What good looks like**

Three exercises, one arc: plan → gap-check → execute. The Lore card is live. Real entries from your show are in the database.

**Why this matters**

The lore entries you added in 2.3 become the target for Elena's accuracy-check prompt in Module 3 — Module 2 output feeds Module 3 input.

## Keep the Foundation Evergreen

### You just added new files, routes, and a page.

The codebase changed. The source of truth hasn't caught up yet.

**ARCHITECTURE.md**
Needs updating

The Lore card, `/lore` page, and `lore_entries` table are new. Future planning prompts won't know about them unless the architecture doc reflects them.

**copilot-instructions.md**
Needs updating

Capture any new naming conventions or patterns from this module — so every future prompt, skill, and agent inherits current practice.

**In Module 3, Exercise 3.1 — you'll build `refresh-docs.prompt.md`**

One reusable prompt that keeps both files current after any meaningful code change. For now, update them **using Copilot Chat**.

SDP



# Module 2 Is Locked In

📝 Next Up: Module 3 — Custom Prompts

🗺️

### You Mapped the Work

Planning now happens before code instead of during implementation thrash

🧠

### You Added Judgment

Architecture, tests, and edge cases are challenged before execution starts

📝

### Now Capture the Pattern

Module 3 turns repeatable planning requests into reusable custom prompts

Better planning makes execution calmer. **Custom prompts make that better workflow repeatable for the whole team.**

Reflect & Discuss

## STORY MOMENTS

The plan listed 14 files — nobody expected that many. The number made the scope real before a single line was written

The gap-finding moment: plan mode surfaced missing pieces the team hadn't discussed — caught before the keyboard, not after

The handoff payoff: a structured plan handed to the implementation agent felt different from winging a prompt — the output reflected it

### ONE BIG QUESTION

**When you get back to your desk, what's the first thing you'd use plan mode on — and what's your guess for how many files it will touch?**

## 💭 The Team, After Module 2

✓ AFTER THE MODULE

### Jessica

*The Eager Junior*

*"It doesn't write slowly. It doesn't write at all yet — it thinks first. I have a whole roadmap before touching a single file. That's not slower. That's completely different."*

Found the workflow nobody put in the onboarding docs.

### David

*The Seasoned Architect*

*"It doesn't output a diagram. It surfaces assumptions before they become implementation constraints. That's more useful than a diagram. I'll revise my position."*

Position revised. Without being asked.

### Sarah

*The Skeptical Senior*

*"It's not read mode. It flagged a dependency conflict and found the right files before a line was written. A junior dev who actually does that would be a very different conversation."*

The bar moved. The numbers moved with it.

### Elena

*The Quality Champion*

*"It does wait before writing — I was right about that. What I didn't expect: it also admits what it doesn't know. That's the part that matters."*

Right about one thing. The other thing was better.

### Marcus

*The DevOps Developer*

*"That's not a diff preview. It planned the whole feature — the files, the order, the dependencies. I was not already using that. I was using something much smaller."*

Found out what he wasn't using. Immediately wanted more.

### Rafael

*The Product Visionary*

*"Marcus did not tell me that. What it actually does is turn 'let's see what happens' into a sequenced delivery roadmap. That's not a ticket description. That's an entire sprint plan."*

Corrected the source. Upgraded the outcome.